COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010- Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	9/1/26	**DATE SUBMITTED:**	9/1/26
NAME:	Paterno Santos Gabriel Emman Serrano Vince Andrew Urbano Ellaiza Vizcayno Jared Isaiah Dela Cruz	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 7.1 **Using Sorting Algorithm**

1. PROCEDURE OUTPUT

1. What is a sorting algorithm?
   - Sorting algorithms are used to rearrange arrays or list of elements in an order. The algorithms can also differ by their output requirements, it can be stable or not stable. Stable sorting means maintaining the original order of equal elements. These sorting tools work for almost anything. You can pick how to sort your items, like smallest to largest, largest to smallest, or by a specific detail like sorting people by age.

2. Where can sorting algorithms be used?

   - Sorting Algorithms can be used in various ways. To name a few, arrange products while online shopping from cheapest to most expensive, sort rankings of students based on grades, arrange a deck of cards in order, organize banking transactions by amount so records are easier to go through, etc.

3. Explain the different types of sorting algorithms.

- **COMPARISON-BASED SORTING ALGORITHM**
  - This type of sorting algorithm determines the correct order of a collection by evaluating elements directly against one another using comparison operators like less-than or greater-than. Their primary use is that they work generically on any data type where a relative order between two items can be defined. The techniques in this category vary from given techniques like Selection Sort, Bubble Sort, and Insertion Sort

- **NON-COMPARISON-BASED SORTING ALGORITHM**
  - This type of algorithm avoids comparing elements directly to one another. Instead, they rely on specific computational methods like array indexing, key distribution, or counting digit values to determine an element's correct position. Since they bypass element comparisons, these algorithms often achieve faster execution times under specific

ACTIVITY NO. 7.1	Using Sorting Algorithm
1. PROCEDURE OUTPUT	

conditions. Counting Sort, Radix Sort, Bucket Sort, and Pigeonhole Sort are some of the examples of this kind of sorting algorithm.

- **HYBRID SORTING ALGORITHM**
  - Hybrid sorting algorithms combine two or more distinct sorting methods to maximize the strengths of each algorithm across different scenarios. They adapt their strategy live based on the size of the dataset or the execution depth to balance time and space efficiency. The main example of this is the IntroSort, which switches between Quick Sort, Heap Sort, and Insertion Sort, and TimSort, which is a combination of Merge Sort and Insertion Sort.

4. Give sample programs in C++ that use sorting algorithms, specifically selection sort, insertion sort, and bubble sort. Explain how the programs work.

Selection Sort

Sorting Algorithms:	Explanation:

``cpp
// C++ program to implement Selection Sort
#include <bits/stdc++.h>
using namespace std;
 	
void selectionSort(vector<int> &arr) {
    int n = arr.size();
 	
    for (int i = 0; i < n - 1; ++i) {
 	
        // Assume the current position holds
        // the minimum element
        int min_idx = i;
 	
        // Iterate through the unsorted portion
        // to find the actual minimum
        for (int j = i + 1; j < n; ++j) {
            if (arr[j] < arr[min_idx]) {
 	
                // Update min_idx if a smaller
                // element is found
                min_idx = j;
            }
        }
 	
        // Move minimum element to its
        // correct position
        swap(arr[i], arr[min_idx]);
    }
}
``
	The selection sort declares a variable that will store the index of the smallest element. Think of it this way, imagine finding the shortest length and then putting it in the front then checking another one for who is the next shortest. The code uses a for loop to go through the unsorted elements, while another for loop compares each element to find the smallest value. Now once the smallest is found, it gets swapped with the element in the current position until all elements are fully sorted. The time complexity of this code is O(n^2) for the worst and the best because it uses a nested loop to compare the elements.

ACTIVITY NO. 7.1 Using Sorting Algorithm

1. PROCEDURE OUTPUT

```
void printArray(vector<int> &arr) {
    for (int &val : arr) {
        cout << val << " ";
    }
    cout << endl;
}

int main() {
    vector<int> arr = {64, 25, 12, 22, 11};

    cout << "Original array: ";
    printArray(arr);

    selectionSort(arr);

    cout << "Sorted array: ";
    printArray(arr);

    return 0;
}
```

ACTIVITY NO. 7.1 **Using Sorting Algorithm**

1. PROCEDURE OUTPUT

Insertion Sort

```cpp
// C++ program for implementation of Insertion Sort
#include <iostream>
using namespace std;

/* Function to sort array using insertion sort */
void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;

        /* Move elements of arr[0..i-1], that are
           greater than key, to one position ahead
           of their current position */
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

/* A utility function to print array of size n */
void printArray(int arr[], int n)
{
    for (int i = 0; i < n; ++i)
        cout << arr[i] << " ";
    cout << endl;
}

// Driver method
int main()
{
    int arr[] = { 12, 11, 13, 5, 6 };
    int n = sizeof(arr) / sizeof(arr[0]);

    insertionSort(arr, n);
    printArray(arr, n);

    return 0;
}
```

The insertion sort first declares a variable to store the current element of the array called the key. This is then put in a for loop to go through the elements starting from the second index, while the while loop compares the key with the elements before it. If the previous element is greater than the key, it gets shifted one position to the right until the key is placed in its correct position. This continues until all elements are fully sorted. The best-case time complexity is O(n) when the list is already sorted, while the worst case is O(n^2).

ACTIVITY NO. 7.1	Using Sorting Algorithm
1. PROCEDURE OUTPUT	

Sorting Algorithms:	Explanation:
Bubble Sort	The bubble sort first declares a variable to store the size of the array to be sorted. This is then put in a for loop to go through every index and the if statement compares two elements and when they are in the wrong order, their indexes get swapped. This continues from left to right and repeats the steps if all elements are fully sorted.

ACTIVITY NO. 7.1 **Using Sorting Algorithm**

1. PROCEDURE OUTPUT

```cpp
// C++ program for the implementation of Bubble sort
#include <bits/stdc++.h>
using namespace std;

void bubbleSort(vector<int>& v) {
    int n = v.size();

    // Outer loop that corresponds to the number of
    // elements to be sorted
    for (int i = 0; i < n - 1; i++) {

        // Last i elements are already
        // in place
        for (int j = 0; j < n - i - 1; j++) {

            // Comparing adjacent elements
            if (v[j] > v[j + 1])

                // Swapping if in the wrong order
                swap(v[j], v[j + 1]);
        }
    }
}

int main() {
    vector<int> v = {5, 1, 4, 2, 8};

    // Sorting the vector v
    bubbleSort(v);
    for (auto i : v)
        cout << i << " ";
    return 0;
}
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

References:

ACTIVITY NO. 7.1 **Using Sorting Algorithm**

1. PROCEDURE OUTPUT

Sorting Algorithms/Types of Sorting Algorithms:
https://www.geeksforgeeks.org/dsa/sorting-algorithms/

Selection Sorting:
https://www.geeksforgeeks.org/dsa/selection-sort-algorithm-2/

Insertion Sorting:
https://www.geeksforgeeks.org/dsa/insertion-sort-algorithm/

Bubble Sorting:
https://www.geeksforgeeks.org/cpp/bubble-sort-in-cpp/

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.